phase_08_B_GAP_AUDIT

Phase 8B Gap Audit — Reverse Integration (Chutes AI / Unified GPU Consumer)

Field	Value
Audited	2026-06-01
Auditor	Engineering auditor (code-grounded)
Scope	`docs/research/PHASE_8B_ROADMAP.md` deliverables vs. actual `pkg/`, `internal/`, `cmd/`, `api/`, `security/`

Honest Completion: ~8% complete

One-line summary: Phase 8B's *crypto foundation* is real and well-tested (ML-KEM-768 E2EE, software TEE attestation, and software GraVal proof-of-GPU all exist in the `security/` submodule), but **every Phase 8B-specific deliverable — the `GPUProvider` interface, `PoolManager`, all four provider adapters, `BurstController`, `ComputeBroker`, CUDA proxy, and all three `cmd/` binaries — is MISSING**. No remote GPU is consumed anywhere in the codebase; `internal/gpu` is strictly local detection/allocation.

3-Axis Package Status (Refreshed 2026-06-04, HXC-939)

> **Why this section exists:** "exists" ≠ "used". `wired` is **measured** via `go list -deps ./cmd/... | grep -Fx <module-path>/<pkg>` (module = `github.com/HelixDevelopment/helix_cluster`), not assumed. **"Completed (registry) ≠ wired"** — Completed only means source+tests exist; it does NOT prove a shipped binary reaches it. `security/pkg/*` is a **separate module** and so never appears in HelixCluster's `cmd/` dep graph.

> **MAJOR STALENESS CORRECTION:** the original 2026-06-01 table marked nearly all of Phase 8B MISSING. That is now **wrong in both directions**. As of 2026-06-04, the three binaries **and** the whole package set **EXIST and are TESTED** — `cmd/gpu-pool-manager`, `cmd/burst-controller`, `cmd/e2ee-proxy`, `pkg/pool`, `pkg/burst`, `pkg/local`, `internal/costbroker`, and all four `pkg/provider/{chutes,ionet,runpod,aws}` adapters. **However, every one of those value packages is ORPHANED — not reachable from any `cmd/` binary's dependency graph** (the binaries do not yet import them). So the registry would call these "Completed" while they are exists-but-unused. The rows below are authoritative over the older prose.

Package / binary	implemented	wired (reachable from `cmd/`)	tested
`cmd/gpu-pool-manager`	yes	yes (is a `main`)	yes
`cmd/burst-controller`	yes	yes (is a `main`)	yes
`cmd/e2ee-proxy`	yes	yes (is a `main`)	yes
`pkg/pool`	yes	**NO (orphaned)**	yes
`pkg/pool/scheduler`	**NO (subdir absent)**	n/a	n/a
`pkg/provider/chutes`	yes	**NO (orphaned)**	yes
`pkg/provider/ionet`	yes	**NO (orphaned)**	yes
`pkg/provider/runpod`	yes	**NO (orphaned)**	yes
`pkg/provider/aws`	yes	**NO (orphaned)**	yes
`pkg/burst`	yes	**NO (orphaned)**	yes
`pkg/local` (LocalGPURegistrar)	yes	**NO (orphaned)**	yes
`internal/costbroker`	yes	**NO (orphaned)**	yes
`pkg/proxy` (CUDA interceptor)	**NO (absent)**	n/a	n/a
`pkg/gpu` (top-level adapter hooks)	**NO (only `internal/gpu`)**	n/a	n/a
`security/pkg/e2ee` (separate module)	yes	**NO (not in helix cmd graph)**	yes
`security/pkg/gpuattest` (separate module)	yes	**NO (not in helix cmd graph)**	yes
`security/pkg/attestation` (separate module)	yes	**NO (not in helix cmd graph)**	yes
	yes	yes	yes

Package / binary	implemented	wired (reachable from cmd/)	tested
internal/gpu (local-only, no tier/ provider)			

Callout (textbook double-orphan): the three cmd/∗ binaries are wired *as binaries*, and the pool/burst/provider/costbroker packages are implemented+tested — but the binaries and the value packages are **not connected to each other** (go list -deps ./cmd/gpu-pool-manager does not include pkg/pool). Both halves look "done" in isolation; the feature is non-functional because the wiring between them is missing. The old "all MISSING" verdict and any "Completed" registry entry are equally misleading — only the measured wired column reveals the truth.

Evidence-Backed Anti-Bluff Notes

- The crypto primitives are genuine: security/pkg/e2ee/package.go imports stdlib crypto/mlkem (ML-KEM-768, FIPS 203) + crypto/hkdf + AES-256-GCM, with 14 real tests (package_test.go). This satisfies the *cryptographic core* of P8B's pkg/e2ee but is **not** wired into any proxy, provider, or outbound-traffic path — so the roadmap deliverable "E2EEProxy for outbound remote-GPU traffic" is PARTIAL at best (primitive present, integration absent).
- security/pkg/gpuattest/package.go self-declares an HONEST SCOPE NOTE: it is *software* HMAC-SHA256 challenge/response, NOT a CUDA/ OpenCL GraVal kernel and NOT hardware-rooted. It maps to the roadmap's GraValVerifier interface but the real GPU-kernel verification is explicitly DEFERRED (hardware required).
- pkg/scheduler/cost_gpu.go and pkg/fiber/fiber.go are tagged "Phase 8C" in their own headers — they are intra-cluster node scoring and a miner□validator framing transport, NOT the P8B cross-provider ComputeBroker or remote spillover. Counting them toward 8B would be a scope bluff.

Deliverable Status Table

Deliverable (roadmap §4/§6)	Status	Evidence file:line	Notes
pkg/pool — GPUProvider iface + PoolManager (P0 #1)	MISSING	no pkg/pool dir (find on pkg/internal/cmd empty)	Single integration seam for all of 8B; nothing exists
pkg/pool/scheduler — Priority/CostAware/ LatencyAware	MISSING	absent	—
pkg/provider/chutes — ChutesProvider (P0 #2)	MISSING	no chutes/io.net ref in pkg/internal/cmd/api	Primary remote burst target; no API client
pkg/provider/ionet — Ray adapter (P1 #6)	MISSING	absent	—

Deliverable (roadmap §4/§6)	Status	Evidence file:line	Notes
pkg/provider/runpod — serverless + warm pool (P1 #7)	MISSING	absent	—
pkg/provider/aws — EC2 Spot (P2 #11)	MISSING	absent	—
pkg/e2ee — ML-KEM-768 crypto core	PARTIAL	security/pkg/e2ee/package.go:18-30, transport.go:15	Real ML-KEM-768+AES-GCM Session + framed Transport, 14 tests; lives in submodule, NOT exposed as pkg/e2ee nor wired to any provider
pkg/e2ee — E2EEProxy (outbound remote-GPU proxy)	MISSING	e2ee/transport.go has no listener/dial/http	Only an io.ReadWriter framing layer; no transparent proxy, no outbound routing
pkg/e2ee — GraValVerifier admission gate (P1 #10)	PARTIAL	security/pkg/gpuattest/package.go:215-282	Software Verifier/Prover + RegisterDevice/Challenge/Verify with 13 tests; software-only (self-declared), NOT a provider-admission gate, no graval.verified label flow
pkg/e2ee — AttestationVerifier (TDX)	PARTIAL	security/pkg/attestation/attestation.go:25-45	Real ed25519 software TEE doc verify (18 tests); hardware TDX quote DEFERRED per its own scope note
pkg/burst — BurstController state machine (P0 #5)	MISSING	no burst ref anywhere	Core value prop (90% spill / 63% recover hysteresis) absent
pkg/proxy — CUDA interceptor + virtual /dev/nvidia* (P1 #8)	MISSING	absent	Requires CUDA/driver work; see DEFERRED
pkg/local — LocalGPURegistrar (TCO cost) (P0 #4)	PARTIAL	internal/gpu/manager.go:14-40, gpu.go	Local GPU detection/alloc/health exists (real / proc on Linux,

Deliverable (roadmap §4/§6)	Status	Evidence file:line	Notes
			mock else); NO TCO/effective-cost model, NO tier registration into a pool
`internal/gpu` — tier mgmt / provider registration / health wiring	PARTIAL	`internal/gpu/manager.go`, `monitor.go`	Local-only Manager+Monitor; zero tier/provider/remote concepts (grep: only "pool" as English word)
`internal/costbroker` — `ComputeBroker` real-time scoring	MISSING	absent	`pkg/scheduler/cost_gpu.go` is Phase 8C intra-node scoring, not multi-provider broker
`cmd/gpu-pool-manager` (gRPC+HTTP front-end)	MISSING	not in `cmd/` listing	—
`cmd/burst-controller`	MISSING	not in `cmd/` listing	—
`cmd/e2ee-proxy`	MISSING	not in `cmd/` listing	—
`pkg/gpu` — `ProviderAdapter` registration hooks	MISSING	no top-level `pkg/gpu` (only `internal/gpu`)	Roadmap names `pkg/gpu`; only `internal/gpu` exists, no adapter hooks
`pkg/metrics` — GPU-tier util / cost-per-hour / provider health	MISSING	`pkg/metrics/` has no tier/cost/provider gauges	Generic metrics only
`pkg/scheduler` — `GPUTier`-aware filter predicate	MISSING	`pkg/scheduler/cost_gpu.go:50` filters on GPU count only	No tier predicate
Helm chart + Prometheus/Grafana 8B dashboards (P1 #9)	MISSING	no 8B pool/burst dashboards under `deploy/`	—
HelixQA `BurstChallenge` (§8 exit gate)	MISSING	no burst/failover challenge in `challenges/`	Mandatory CLAUDE-1 gate unmet
E2E `helix submit` → saturate → route-to-Chutes demo	MISSING	no remote routing path	Mandatory CLAUDE-1 gate unmet
Real-provider CI integration (Chutes/ io.net/RunPod)	MISSING	no provider client to integration-test	Mandatory "no mock-only" gate unmet

TOP IMPLEMENTABLE GAPS (Go, no new infra)

These are concrete, no-new-hardware, no-new-cluster-infra deliverables, in dependency order. Each is mutation-pairable per §1.1.

1. `pkg/pool` — `GPUProvider` interface + in-memory `PoolManager` (P0 #1)

Target dir: `pkg/pool/` **Spec:** Define the single integration seam: `GPUProvider` interface (`Name() string`, `Tier() Tier`, `Probe(ctx) (Capacity, error)`, `Submit(ctx, WorkloadRequest) (Result, error)`, `Healthy(ctx) bool`), plus value types `VirtualGPU`, `WorkloadRequest`, `Capacity`, and a `PoolManager` that registers providers, sorts them by `Tier` priority (Local=1…Decentralized=4), and on `Allocate(req)` walks tiers in order returning the first healthy provider with capacity. Pure Go, in-memory registry, deterministic ordering, context-cancellable. No network calls — providers are interfaces, tested with a `FakeProvider`. **Acceptance test (mutation-pairable):** With three registered fake providers (tiers 1/3/4) where tier-1 reports zero capacity and tier-3 healthy, `Allocate` MUST return the tier-3 provider's result. Mutation: flip the tier-sort comparator to descending → test must fail (would pick tier-4). Second mutation: skip the capacity check → test must fail (would pick saturated tier-1).

2. `internal/costbroker` — `ComputeBroker` deterministic scorer (internal/costbroker)

Target dir: `internal/costbroker/` **Spec:** Implement the roadmap's weighted scorer: given a slice of `ProviderSignal{Price, Availability, Latency, Throughput}`, compute `score = 0.30*priceScore + 0.30*availability + 0.20*latencyScore + 0.20*throughput` (price/latency inverted+normalized to [0,1]). Expose `Rank([]ProviderSignal) []Ranked` returning a stable, descending ordering, and enforce a hard `MaxCostPerHour` cap that drops over-budget providers entirely. Re-rank is pure/stateless so it can be called every 60s by a caller. No external data — signals are injected. **Acceptance test (mutation-pairable):** Given a cheap-but-slow provider vs. an expensive-fast one with documented weights, assert the exact ranking and that a provider above `MaxCostPerHour` is excluded. Mutation: change a weight constant (0.30→0.50) → ordering assertion must flip and fail. Mutation: remove the cap filter → excluded provider reappears and the count assertion fails.

3. `pkg/burst` — `BurstController` hysteresis state machine (P0 #5)

Target dir: `pkg/burst/` **Spec:** A clock-injectable state machine with states `MONITOR → SPILL → RECOVER`. Input is a utilization sample `[0,1]`; transitions: ≥0.90 enters SPILL (emits a spill event), and only drops back to MONITOR when utilization falls to ≤0.63 (hysteresis — no flapping between). Expose `Observe(util float64) (Transition, []Event)` and a queryable `State()`. No goroutines required for the core (caller drives ticks); fully deterministic. **Acceptance test (mutation-pairable):** Feed the sequence 0.5→0.95→0.80→0.62 and assert exactly one SPILL at 0.95 and one RECOVER at 0.62 (the 0.80 sample stays in SPILL due to hysteresis). Mutation: collapse the two thresholds to a single

0.90 boundary → the 0.80 sample wrongly recovers and the "exactly one RECOVER" assertion fails. Mutation: make recover use >= 0.63 → 0.80 triggers recover, test fails.

4. `pkg/provider/chutes` — `ChutesProvider` OpenAI-compatible adapter (httptest) (P0 #2)

Target dir: `pkg/provider/chutes/` **Spec:** Implement `GPUProvider` over an OpenAI-compatible `/v1/chat/completions` REST endpoint: configurable base URL + API key, JSON request/response marshaling, HTTP-429 honoring with exponential backoff (capped), context deadline propagation, and a `Healthy` probe via a lightweight GET. The HTTP client is injectable so it is tested against an `httptest.Server` (real HTTP round-trip, not a mock interface). Live-endpoint CI remains DEFERRED until a Chutes sandbox key exists, but the wire-correct adapter is fully buildable now. **Acceptance test (mutation-pairable):** Drive against an `httptest.Server` that returns 429 once then 200 with a known completion; assert the adapter retried and returned the parsed content, and that an injected fake clock shows backoff was applied. Mutation: remove the 429 retry branch → request fails on first 429 and content assertion fails. Mutation: ignore the response body field mapping → parsed content mismatch fails.

5. `pkg/local` — `LocalGPURegistrar` with TCO effective-cost (P0 #4)

Target dir: `pkg/local/` (wrapping `internal/gpu`) **Spec:** Wrap the existing `internal/gpu.Manager` inventory and produce `VirtualGPU` entries tagged `Tier=Local` with an *effective* `$/hr` derived from a TCO model: `effectiveCost = (capexAmortizedPerHour + powerWatts*pricePerKWh/1000) / utilizationFactor`, all inputs from config (no invented external prices). Registers the local tier into a `pkg/pool.PoolManager`. This is the pre-condition the roadmap names ("pool must know local tier before spillover"). **Acceptance test (mutation-pairable):** Given a fixed GPU count and config (capex, watts, kWh price, util), assert the computed effective `$/hr` equals a hand-derived constant and that exactly N local `VirtualGPU`s register at Tier=Local. Mutation: drop the power term from the cost formula → numeric assertion fails. Mutation: tag tier as Remote → tier assertion fails.

6. `cmd/gpu-pool-manager` — HTTP front-end over `pkg/pool` (cmd binary)

Target dir: `cmd/gpu-pool-manager/` **Spec:** Thin binary exposing `pkg/pool.PoolManager` over HTTP: `POST /allocate` (WorkloadRequest → chosen provider+result), `GET /providers` (registered providers + tier + health), `GET /healthz`. Reuses existing repo conventions (`pkg/log`, graceful shutdown like other `cmd/helix-*`). No business logic in the binary — it wires pool + costbroker + local registrar. **Acceptance test (mutation-pairable):** `httptest` the handler with two fake providers; `POST /allocate` returns the tier-correct provider as JSON and `GET /providers` lists both with health. Mutation: have the handler ignore tier ordering from the pool → allocate returns wrong provider, JSON assertion fails.

DEFERRED (external infra / hardware / non-Go)

Item	Reason
Real GraVal CUDA/OpenCL GPU kernel verification	Requires NVIDIA GPU + CUDA toolchain; `security/pkg/gpuattest` is software-only by explicit design and stands in as the interface seam
Intel TDX attestation enforcement (real quotes)	Requires TDX-capable hardware + vendor cert chain; `security/pkg/attestation` ships a software ed25519 stand-in only
`pkg/proxy` CUDA API interceptor + virtual `/dev/nvidia*`	Requires CUDA driver shimming, kernel/device-node creation, Linux GPU host — not implementable as pure portable Go; gRPC kernel-forwarding also needs a remote GPU host
Live-endpoint CI vs Chutes / io.net / RunPod / AWS Spot	Requires real provider accounts/credits + network egress; adapters can be built and httptest-verified now, live integration gated on credentials
`pkg/provider/aws` EC2 Spot interrupt + CRIU/S3 checkpoint	Needs AWS account, Spot fleet, and CRIU host support
48-hour load test / 7-day soak / chaos suite (§8 KPIs)	Requires sustained real infra + GPU fleet; cannot run in unit/CI alone
Key rotation CronJob + HSM	Needs Kubernetes CronJob runtime + HSM device
Grafana sink-side burst screenshots (§8 gates)	Requires a running Prometheus/Grafana + live burst event to capture

Bottom Line

The reusable security crypto (ML-KEM-768 E2EE, software attestation, software GraVal) is real and tested but sits in the `security/` submodule as primitives, not as Phase 8B components. The entire reverse-integration mechanism — provider abstraction, pool manager, cost broker, burst controller, and the three binaries — is unbuilt. Items 1–6 above are the honest, no-new-infra critical path to a functioning 8B MVP skeleton.